

Music Genre Classification on FMA Datasets

61.502 Deep Learning for Enterprise (Y2026)

We built a CNN-based audio classification pipeline on FMA-Large, compared model variants under a reproducible protocol, and packaged the result into a working demo flow.

Ang Kar Kin Nicolas Lasch Nisal Jinadasa

April 2026

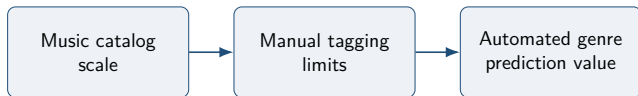
System Overview

- FMA metadata + audio
- Mel features + CNN training
- Evaluation + demo API

Placeholder

Repo / demo link placeholder

- Supports playlisting, search, recommendation, and catalog organization.
- Manual tagging is expensive and inconsistent at large scale.
- Genre labels are proxy labels, not ground truth, so overlap is expected.
- Audio varies over time, so short clips may miss the full track identity.
- This project treats the task as an end-to-end ML pipeline, not a notebook demo.



Framing: useful for retrieval, discovery, and downstream personalization.

Why Discuss Both?

- FMA-Medium is a standard benchmark subset.
- FMA-Large increases label diversity and scale.
- Comparing both helps explain why this project chose the harder setup.

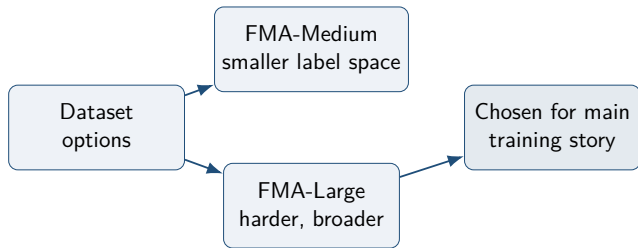
Source: official FMA dataset release / ISMIR 2017 paper.

Dataset	Tracks	Genres
FMA-Medium	25,000	16
FMA-Large	106,574	161

FMA-Medium
cleaner benchmark

FMA-Large
higher-scale stress test

- FMA-Medium offers a simpler benchmark with fewer classes.
- FMA-Large better reflects scale, imbalance, and ambiguity in realistic music catalogs.
- This repo trains and evaluates on `fma_large` using labels from `fma_metadata/tracks.csv`.
- The main reported task is single-label top-genre classification.



- Keep only subset == large and non-null `track.genre_top`.
- Exclude missing or known-bad audio where needed.
- Use a stratified 80 / 10 / 10 train/val/test split.
- Apply a rare-class support policy before final training.

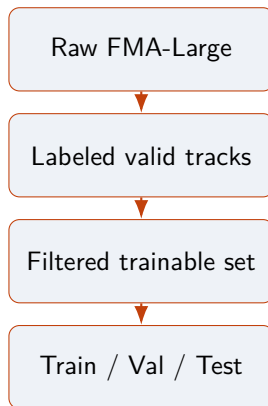
Main Run Counts

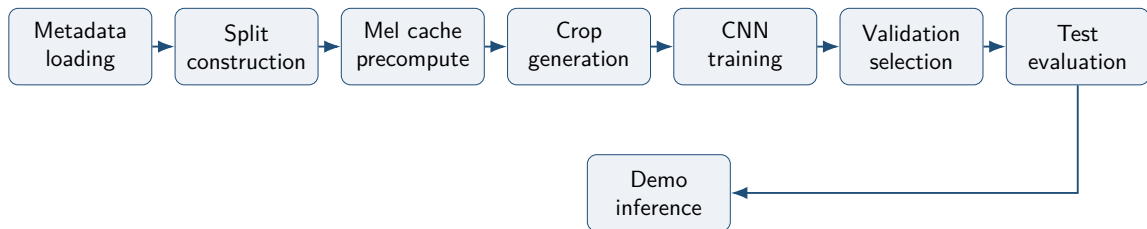
Labeled valid tracks: **24,552**

Train / Val / Test: **19,549 / 2,444 / 2,444**

Final classes after support policy: **11**

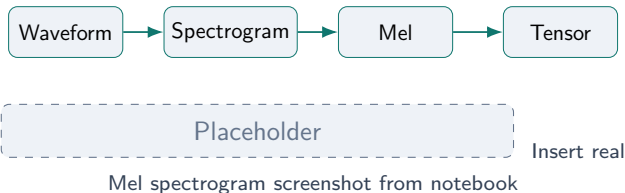
Support-policy thresholds in the notebook reduce the final label space to 11 classes.





- The main training story and the later demo share the same high-level pipeline shape.
- The anchor idea is reproducibility: consistent preprocessing, comparable runs, and saved artifacts.

- Audio is standardized to **22,050 Hz**.
- Features use **128 Mel bins** with `n_fft = 2048` and `hop = 512`.
- The pipeline precomputes up to **29 s** full-track Mel caches.
- Training and evaluation use **5 s** crops for a fixed model input view.
- Mel features preserve useful structure while keeping training practical.



Standard CNN

- Simpler stacked convolution blocks
- Baseline and augmentation comparison runs
- Lower capacity and weaker stability



Residual CNN

- Residual blocks improve gradient flow
- Strongest model in the saved results
- Better optimization stability on the harder setup



- **AdamW**, 35 epochs, batch size 64
- Balanced sampler and focal loss for imbalance
- Augmentation enabled for selected runs
- Early stopping on validation macro-F1
- Multi-crop validation and test evaluation

Optimizer
AdamW

Data balance
sampler + focal loss

Selection
val macro-F1

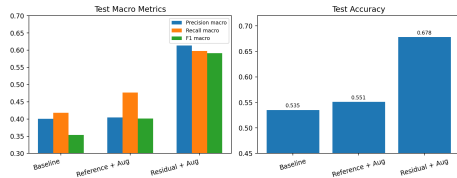
Augmentation
selected runs

Evaluation
multi-crop

Why: the protocol aims to make model comparisons defensible rather than optimistic.

Model	Val F1	Test Acc	Test Rec	Test F1
standard_base	0.3624	0.5352	0.4180	0.3536
residual	0.5733	0.6780	0.5974	0.5909
standard+aug	0.4141	0.5511	0.4771	0.4009

Residual test macro-precision: **0.6135**



- The residual CNN is strongest and most stable across validation and test.
- Macro-F1 is the key metric because minority genres are harder than major classes.
- Weighted metrics are higher, which indicates imbalance is still masking some failures.
- Error analysis should focus on confusions between semantically adjacent genres.

Placeholder

Insert confusion matrix screenshot

Placeholder

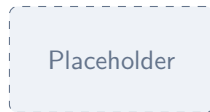
Insert per-class failure examples

- User uploads audio or selects a sample in the frontend.
- The frontend sends the file to a FastAPI backend.
- The API preprocesses audio, runs inference, and returns top predictions.
- The UI renders ranked genres and confidence bars.
- This demo uses a later trained model, so its metrics are reserved separately from the main benchmark story.

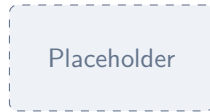


- 1 Pick a sample track or upload a new file.
- 2 Backend resamples, trims or pads, and computes features.
- 3 The model returns top-K predicted genres.
- 4 The frontend renders confidence bars for interpretation.

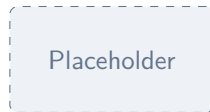
Use this slide to transition cleanly into the live demo.



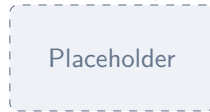
Landing page



Prediction bars

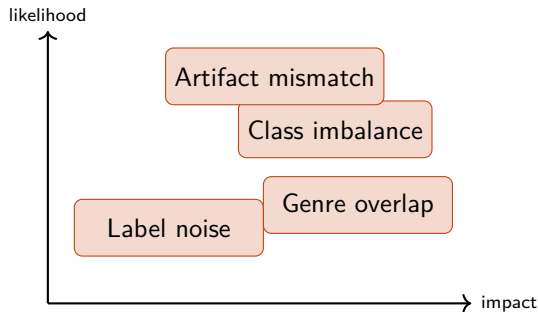


Selected sample / upload



API / network flow

- Genre labels are noisy and sometimes ambiguous.
- Class imbalance still hurts minority-genre performance.
- Adjacent genres remain a major source of confusion.
- The main evaluated model and later demo model are not the same artifact.



- We built a reproducible audio ML pipeline from dataset curation to inference.
- The residual CNN is the best reported model in the main benchmark story.
- The later demo model shows a practical path toward interactive deployment.



Next Step

Unify the production demo with the best evaluated checkpoint and report its deployment metrics.